

6.7 用 Verilog HDL 描述时序逻辑电路

6.7.1 移位寄存器的Verilog建模

6.7.2 计数器的Verilog建模

6.7.3 状态转换图的Verilog建模

6.7.4 数字钟的Verilog建模

6.7.1 移位寄存器的 Verilog 建模

用行为级描述 always 描述一个 4 位双向移位寄存器，有异步清零、同步置数、左移、右移和保持。功能同 74xx194。

```
module shift74x194 (S1, S0, D, Dsl, Dsr, Q, CP, CR);  
    input S1, S0;                // 控制输入  
    input Dsl, Dsr;              // 串行输入  
    input CP, CR;                // 时钟及清零  
    input [3:0] D;               // 并行输入  
    output [3:0] Q;              // 寄存器输出  
    reg [3:0] Q;
```

6.7.1 移位寄存器的 Verilog 建模

```
always @(posedge CP or negedge CR)
```

```
if (~CR) Q <= 4'b0000;
```

```
else
```

```
    case ({S1,S0})
```

```
        2'b00: Q <= Q;           // 保持
```

```
        2'b01: Q <= {Q[2:0],Dsr}; // 右移
```

```
        2'b10: Q <= {Dsl,Q[3:1]}; // 左移
```

```
        2'b11: Q <= D;           // 并行输入
```

```
    endcase
```

6.7.2 计数器的 Verilog 建模

用 Verilog 描述带使能端和同步置数端的可逆 4 位二进制计数器

```
module updowncount_beh #(parameter n=4)
(
input Load,Up_down,En,CP, // 输入端口声明
input[n-1:0]D,           // 并行数据输入
output reg[n-1:0]Q  // 数据输出端口及变量数据类型声明
);
integer direction; // 中间变量声明
```

always @(posedge CP)

begin

if(Up_down)

direction <= 1;

else

direction <= -1;

if(Load)

Q <= D; //Load=1, 同步置数

else if(En)

Q <= Q+direction; // 加 1 或减 1

else

Q <= Q; // 输出保持不变

end

endmodule

6.7.3 状态转换图的 Verilog 建模

用 Verilog 描述状态转换图非常方便，常用 `always` 或 `case` 语句。

下面程序，第一个 `always` 语句描述状态转移

```
module Mealy_sequence_detector (A, CP, CR, Y);
```

```
input A, CP, CR;
```

```
output Y;
```

```
reg Y;
```

```
reg [1:0] current_state, next_state;
```

```
parameter S0=2'b00, S1=2'b01, S2=2'b11;
```

```
always @( negedge CP or negedge CR)
```

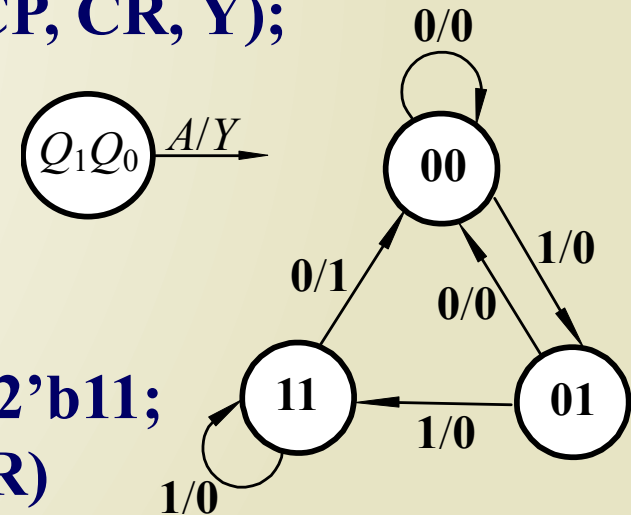
```
begin
```

```
    if (~CR) current_state <= S0;    // 在 CR 下降沿设 s0 为
```

初态

```
    else      current_state <= next_state;
```

```
end
```



6.7.3 状态转换图的 Verilog 建模

第二个 always 是将 current_state 和输入 A 作为敏感变量，判断状态转移条件。

```
always @(current_state,A) // 组合电路
```

```
begin
```

```
case(current_state) // 准备下一状态，电路输出
```

```
Y=0;
```

```
S0:begin next_state=(A==1)?S1:S0;end
```

```
S1:begin next_state=(A==1)?S2:S0;end
```

```
S2:if(A==1) begin next_state = S2;end
```

```
else begin Y=1;next_state = S0;end
```

```
default begin next_state = S0; end
```

```
endcase
```

```
end
```

```
endmodule
```

